

Data Preparation and Document Representation

Text Mining

Credits:

Much of material here presented is based on the book "Speech and Language Processing" (Jurafsky and Martin, 2008), its most recent version available online, the Coursera NLP course (Jurafsky and Martin, 2012) and slides by D. Jurafsky. It's also based on several presentations found in the web, including slides from Jean Mark Gawron from San Diego State University. A special thanks to Nuno Mamede and Luisa Coheur from IST, Lisboa.

Contents

- Basic Text Processing
 - Regular Expressions
 - Tokenisation and sentence segmentation
 - Text Normalisation
- Document Representation
 - Documents (text) as vectors
 - Words as vectors

Remember

Features of Text Data

- High dimensionality
- Large number of features
- Multiple ways to represent the same concept
- Highly redundant data
- Unstructured data
 - Easy for humans, hard for a machine
- Abstract ideas hard to represent
- Huge amount of data to be processed
 - Automation is required

Data Preparation: Text Processing

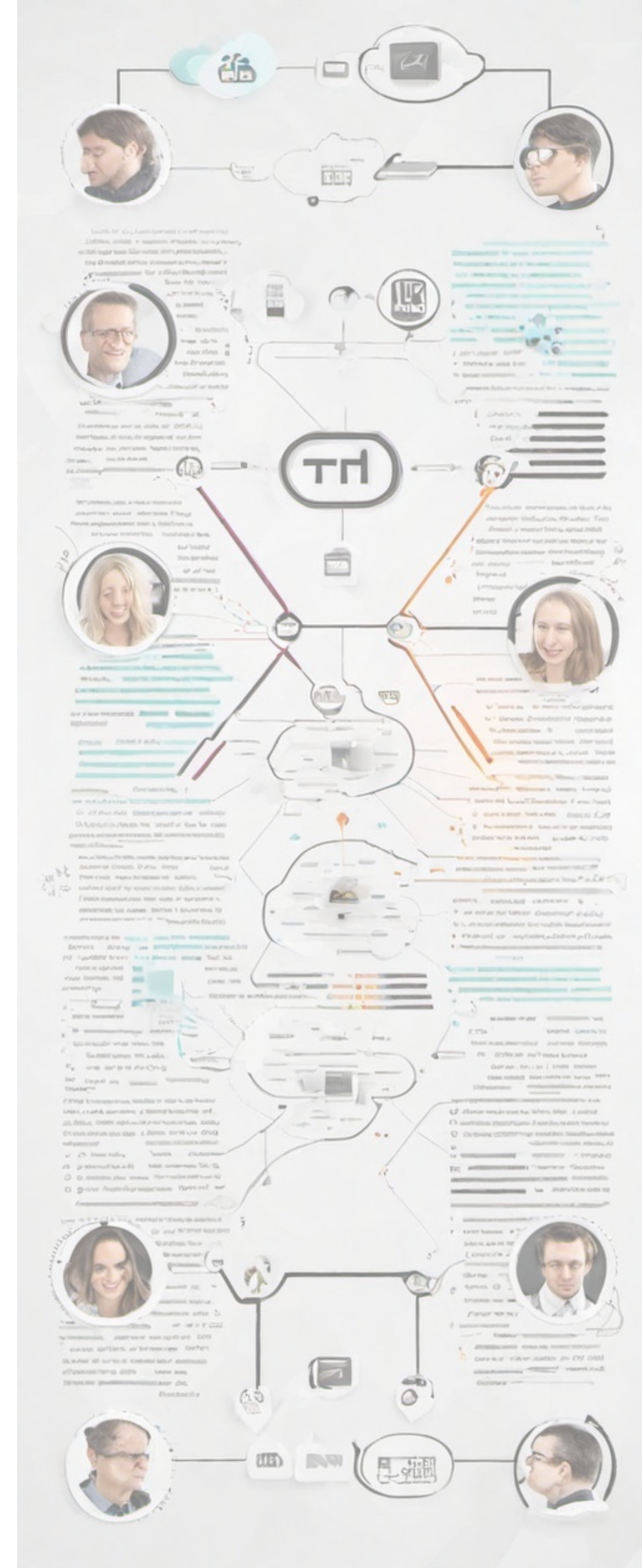
- Basic Text Processing
 - Regular Expressions
 - Word Tokenisation
 - Text Normalisation
 - Case Folding
 - Spell Correction
 - Stemming
 - Sentence Segmentation
- Natural Language Processing
 - Morphologic analysis
 - Lemmatisation
 - Part-of-Speech (POS) Tagging
 - NER - Named Entity Recognition

Data Preparation: Text Processing

- Basic Text Processing
 - Regular Expressions
 - Word Tokenisation
 - Text Normalisation
 - Case Folding
 - Spell Correction
 - Stemming
 - Sentence Segmentation
- Natural Language Processing
 - Morphologic analysis
 - Lemmatisation
 - Part-of-Speech (POS) Tagging
 - NER - Named Entity Recognition

Basic text processing

Regular Expressions



Basic Text Processing

Regular Expressions

- The most basic and fundamental tool for text processing
- A formal language for specifying text strings
- Regular expressions are used everywhere ...
 - Text processing applications: Word, vi, emacs
 - Programming languages: perl, python, java;
 - Shell: grep, sed, awk
- Play a surprisingly large role ... sophisticated sequences of regular expressions are often the **first model for any text processing task**
- We use machine learning classifiers for hard tasks
 - But REs are **used as features** in the classifiers
 - Can be very **useful in capturing generalizations**



Regular Expressions: disjunctions

- Letters inside square brackets: `[]`

Pattern	Matches
<code>[Ll]ibelinha</code>	Libelinha, libelinha
<code>[1234567890]</code>	Any digit

Regular Expressions: disjunctions

- Letters inside square brackets: []

Pattern	Matches
[Ll]ibelinha	Libelinha, libelinha
[1234567890]	Any digit

- Ranges: [A-Z]

Pattern	Matches
[A-Z]	An upper case letter
[a-z]	A lower case letter
[0-9]	A single digit

Regular Expressions: disjunctions

- Negations: `[^Ss]`
 - `^` means negation only when is the first char in `[ ]`

Regular Expressions: disjunctions

- Negations: `[^Ss]`
 - `^` means negation only when is the first char in `[]`

Pattern	Matches
<code>[^A-Z]</code>	Not an upper case letter
<code>[^Ss]</code>	Neither 'S' nor 's'
<code>[^e^]</code>	Neither e nor ^
<code>a^b</code>	The pattern a carat b

Regular Expressions: disjunctions

- Other names for “libelinha” are “Libélula”, “tira-olhos”, “lavadeira”
 - The pipe | is used for such disjunction

Pattern	Matches
<code>libelinha tira-olhos</code>	
<code>libelinha tira-olhos libélula</code>	
<code>[Ll]ibelinha [Tt]ira-olhos</code>	
<code>a b c</code>	<code>= [abc]</code>

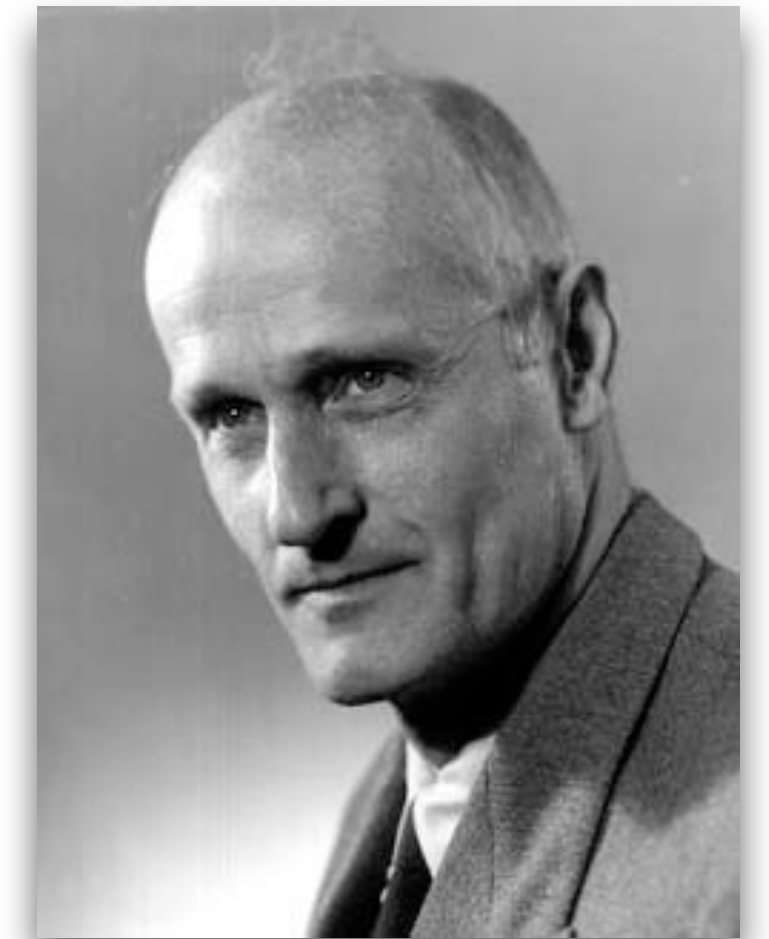
Regular Expressions: repetitions ? * + .

- Repeated symbols
 - Two Kleene characters: Kleene * and Kleene +

Pattern	Matches
<code>colou?r</code>	Optional previous char
<code>oo*h!</code>	0 or more of previous char
<code>o+h!</code>	1 or more of previous char
<code>baa+</code>	e.g. baa baaa baaaa baaaaa
<code>beg.n</code>	e.g. begin begun begun beg3n

- Advanced operators

<code>[0-9]{3}</code>	3 consecutive numbers
<code>[a-z]{3,6}</code>	3 to 6 lowercase letters
<code>[a-z]{3,}</code>	more than 3 lowercase letters



Stephen C. Kleene

Regular Expressions: anchors [^] \$

- Used to mark the *beginning* or the *end* of a line

Pattern	Matches
[^] [A-Z]	<u>A</u> person
[^] [[^] A-Z a-z]	<u>1</u> dog
\ . \$	The end <u>.</u>
. \$	The end <u>?</u> The end <u>!</u>

Regular Expressions: operator precedence

- Operator precedence hierarchy

parenthesis	()
Counters	* + ? { }
sequences and anchors	
Disjunction	

- Examples

`maria+`

`(ah)+`

`(ah){2,}`

`(ah|eh){2,4}`

`([A-Z]{2,}|[0-9]+)`

Regular Expressions:

Filtering and Searching documents

```
ubuntu@hal9000:~$ cat /etc/passwd
```

```
...
```

```
a82950:x:1155:1003:Paulo Rodrigues,, ,Paulo_Rodrigues@iscte-iul.pt:/home/a82950:/bin/bash
```

```
a82962:x:1156:1003:Pedro Santos,, ,Pedro_Couto@iscte-iul.pt:/home/a82962:/bin/bash
```

```
a99660:x:1157:1003:Platthy Agoston,, ,Platthy_Istvan_Agoston@iscte-iul.pt:/home/a99660:/bin/bash
```

```
a78143:x:1158:1003:Rodrigo Ponte,, ,Rodrigo_Alexandre_Ponte@iscte-iul.pt:/home/a78143:/bin/bash
```

```
a94943:x:1159:1003:Tiago Pratas,, ,Emanuel_Pratas@iscte-iul.pt:/home/a94943:/bin/bash
```

```
a91778:x:1160:1003:Carolina Marreiros,, ,Carolina_Marreiros@iscte-iul.pt:/home/a91778:/bin/bash
```

```
jcoelho:x:1161:1003:Joao Coelho,, ,joao.coelho@caixamagica.pt:/home/jcoelho:/bin/bash
```

```
a90022:x:1162:1003:Catia Tavares,, ,Catia_Daniela_Tavares@iscte-iul.pt:/home/a90022:/bin/bash
```

```
aneto:x:1163:1003:Antonio Neto,, ,antonio.neto@caixamagica.pt:/home/aneto:/bin/bash
```

```
beatriz:x:1164:1003:Beatriz Paula,, ,bia.paula11@gmail.com:/home/beatriz:/bin/bash
```

Regular Expressions:

Filtering and Searching documents

```
ubuntu@hal9000:~$ cat /etc/passwd
```

```
...
```

```
a82950:x:1155:1003:Paulo Rodrigues,, ,Paulo_Rodrigues@iscte-iul.pt:/home/a82950:/bin/bash
a82962:x:1156:1003:Pedro Santos,, ,Pedro_Couto@iscte-iul.pt:/home/a82962:/bin/bash
a99660:x:1157:1003:Platthy Agoston,, ,Platthy_Istvan_Agoston@iscte-iul.pt:/home/a99660:/bin/bash
a78143:x:1158:1003:Rodrigo Ponte,, ,Rodrigo_Alexandre_Ponte@iscte-iul.pt:/home/a78143:/bin/bash
a94943:x:1159:1003:Tiago Pratas,, ,Emanuel_Pratas@iscte-iul.pt:/home/a94943:/bin/bash
a91778:x:1160:1003:Carolina Marreiros,, ,Carolina_Marreiros@iscte-iul.pt:/home/a91778:/bin/bash
jcoelho:x:1161:1003:Joao Coelho,, ,joao.coelho@caixamagica.pt:/home/jcoelho:/bin/bash
a90022:x:1162:1003:Catia Tavares,, ,Catia_Daniela_Tavares@iscte-iul.pt:/home/a90022:/bin/bash
beatriz:x:1164:1003:Beatriz Paula,, ,bia.paula11@gmail.com:/home/beatriz:/bin/bash
```

Suppose you want to filter all the **people** containing names
=> with **5 letters** and **ending with “a”, “e” or “o”**

Regular Expressions:

Filtering and Searching documents

```
ubuntu@hal9000:~$ cat /etc/passwd
```

```
...
```

```
a82950:x:1155:1003:Paulo Rodrigues,, ,Paulo_Rodrigues@iscte-iul.pt:/home/a82950:/bin/bash
a82962:x:1156:1003:Pedro Santos,, ,Pedro_Couto@iscte-iul.pt:/home/a82962:/bin/bash
a99660:x:1157:1003:Platthy Agoston,, ,Platthy_Istvan_Agoston@iscte-iul.pt:/home/a99660:/bin/bash
a78143:x:1158:1003:Rodrigo Ponte,, ,Rodrigo_Alexandre_Ponte@iscte-iul.pt:/home/a78143:/bin/bash
a94943:x:1159:1003:Tiago Pratas,, ,Emanuel_Pratas@iscte-iul.pt:/home/a94943:/bin/bash
a91778:x:1160:1003:Carolina Marreiros,, ,Carolina_Marreiros@iscte-iul.pt:/home/a91778:/bin/bash
jcoelho:x:1161:1003:Joao Coelho,, ,joao.coelho@caixamagica.pt:/home/jcoelho:/bin/bash
a90022:x:1162:1003:Catia Tavares,, ,Catia_Daniela_Tavares@iscte-iul.pt:/home/a90022:/bin/bash
beatriz:x:1164:1003:Beatriz Paula,, ,bia.paula11@gmail.com:/home/beatriz:/bin/bash
```

Suppose you want to filter all the **people** containing names
=> with **5 letters** and **ending with “a”, “e” or “o”**

```
ubuntu@hal9000:~$ cat /etc/passwd | grep "[: ][A-Z]...[aoe][ ,]" --color
```

```
...
```

```
a82950:x:1155:1003:Paulo Rodrigues,, ,Paulo_Rodrigues@iscte-iul.pt:/home/a82950:/bin/bash
a82962:x:1156:1003:Pedro Santos,, ,Pedro_Couto@iscte-iul.pt:/home/a82962:/bin/bash
a78143:x:1158:1003:Rodrigo Ponte,, ,Rodrigo_Alexandre_Ponte@iscte-iul.pt:/home/a78143:/bin/bash
a94943:x:1159:1003:Tiago Pratas,, ,Emanuel_Pratas@iscte-iul.pt:/home/a94943:/bin/bash
a90022:x:1162:1003:Catia Tavares,, ,Catia_Daniela_Tavares@iscte-iul.pt:/home/a90022:/bin/bash
beatriz:x:1164:1003:Beatriz Paula,, ,bia.paula11@gmail.com:/home/beatriz:/bin/bash
```

Regular Expressions: example

- Let's try it with an example (regex101.com, regexr.com)

Jogo GTA V rende 800 milhões no dia de lançamento.

O jogo chegou às lojas no dia 17 de Setembro e nas primeiras 24 horas conseguiu arrecadar 800 milhões de dólares, cerca de 589 milhões de euros.

Os dados foram revelados pela Take-Two Interactive Software e revelam que a quinta edição do Grand Theft Auto conseguiu bater o já elevado valor do volume quatro, lançado em 2008.

De acordo com a Forbes, a Rockstar Games, empresa responsável pelo jogo, gastou 266 milhões de dólares com a quinta edição do GTA, exactamente um terço do valor arrecadado com um dia de vendas.

Regular Expressions: errors

- Find me all instances of the word “the” in a text
 - `the` misses capitalised examples
 - `[tT]he` incorrectly returns additional words, like theology or then
 - `[^a-zA-Z][tT]he[^a-zA-Z]` fails to match the first word of a line

Regular Expressions: errors

- Find me all instances of the word “the” in a text
 - `the` misses capitalised examples
 - `[tT]he` incorrectly returns additional words, like theology or then
 - `[^a-zA-Z][tT]he[^a-zA-Z]` fails to match the first word of a line
- In NLP we are always trying to fix two kinds of errors
 - **False positives:** matching strings that we *should not* have matched (there, then, other)
 - **False negatives:** not matching things that *we should* have matched (The)

Regular Expressions: errors

- Find me all instances of the word “the” in a text
 - `the` misses capitalised examples
 - `[tT]he` incorrectly returns additional words, like theology or then
 - `[^a-zA-Z][tT]he[^a-zA-Z]` fails to match the first word of a line
- In NLP we are always trying to fix two kinds of errors
 - **False positives**: matching strings that we *should not* have matched (there, then, other)
 - **False negatives**: not matching things that *we should* have matched (The)
- We’ll be telling the same story for many tasks.
 - Reducing the **error rate** for an application often involves two **antagonistic** efforts
 - Increasing **accuracy** or **precision** (minimizing *false positives*)
 - Increasing **coverage** or **recall** (minimizing *false negatives*)

Regular Expressions: errors

- Find me all instances of the word “the” in a text
 - `the` misses capitalised examples
 - `[tT]he` incorrectly returns additional words, like theology or then
 - `[^a-zA-Z][tT]he[^a-zA-Z]` fails to match the first word of a line
- In NLP we are always trying to fix two kinds of errors
 - **False positives**: matching strings that we *should not* have matched (there, then, other)
 - **False negatives**: not matching things that *we should* have matched (The)
- We’ll be telling the same story for many tasks.
 - Reducing the **error rate** for an application often involves two **antagonistic** efforts
 - Increasing **accuracy** or **precision** (minimizing *false positives*)
 - Increasing **coverage** or **recall** (minimizing *false negatives*)

Doctor (*aka* ELIZA)

- A typical example (Weizenbaum, 1976) of a conversation between a young lady and Doctor (the lady speaks first)

Men are all alike.
In what way?
They're always bugging us about something or other.
Can you think of a specific example?
Well, my boyfriend made me come here.
Your boyfriend made you come here?
He says I'm depressed much of the time.
I am sorry to hear you are depressed.
It's true. I am unhappy.
Do you think coming here will help you not to be unhappy?
I need some help, that much seems certain.
What would it mean to you if you got some help?
Perhaps I could learn to get along with my mother.
Tell me more about your family.
...

Doctor (*aka* ELIZA)

- A typical example (Weizenbaum, 1976) of a conversation between a young lady and Doctor (the lady speaks first)

Men are all alike.
In what way?
They're always bugging us about something or other.
Can you think of a specific example?
Well, my boyfriend made me come here.
Your boyfriend made you come here?
He says I'm depressed much of the time.
I am sorry to hear you are depressed.
It's true. I am unhappy.
Do you think coming here will help you not to be unhappy?
I need some help, that much seems certain.

```
s/I am|I'm/YOU ARE/  
s/my/YOUR/  
...  
s/.* YOU ARE (depressed|sad) .*/I AM SORRY TO HEAR YOU ARE \1/  
s/.* YOU ARE (depressed|sad) .*/WHY DO YOU THINK YOU ARE \1/  
s/.* all .*/IN WHAT WAY/  
s/.* always .*/CAN YOU THINK OF A SPECIFIC EXAMPLE/  
...
```

Doctor (*aka* ELIZA)

- A typical example (Weizenbaum, 1976) of a conversation between a young lady and Doctor (the lady speaks first)

Men are all alike.
In what way?
They're always bugging us about something or other.
Can you think of a specific example?
Well, my boyfriend made me come here.
Your boyfriend made you come here?
He says I'm depressed much of the time.
I am sorry to hear you are depressed.
It's true. I am unhappy.
Do you think coming here will help you not to be unhappy?
I need some help, that much seems certain.

```
s/I am|I'm/YOU ARE/  
s/my/YOUR/  
...  
s/.* YOU ARE (depressed|sad) .*/I AM SORRY TO HEAR YOU ARE \1/  
s/.* YOU ARE (depressed|sad) .*/WHY DO YOU THINK YOU ARE \1/  
s/.* all .*/IN WHAT WAY/  
s/.* always .*/CAN YOU THINK OF A SPECIFIC EXAMPLE/  
...
```

Summary

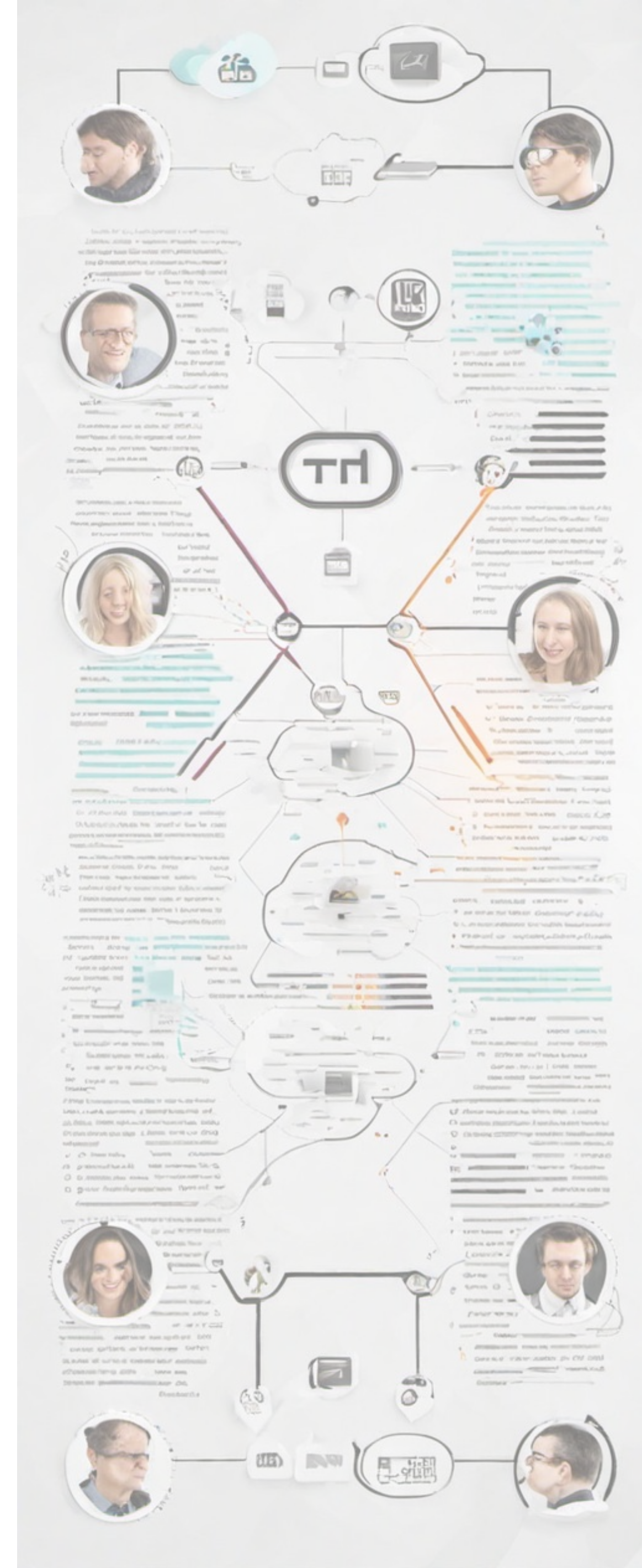
- Regular expressions play a surprisingly large role
 - Sophisticated sequences of regular expressions are often the **first model for any text processing text**
- For many hard tasks, we use machine learning classifiers
 - But regular expressions are **used as features** in the classifiers
 - Can be very **useful in capturing generalizations**

Basic text processing

Tokenisation and sentence segmentation

Word-based and subword tokenization

Sentence segmentation



Text Normalization

- Every NLP task requires text normalization:
 - Tokenizing (segmenting) words
 - Normalizing word formats
 - Segmenting sentences

Space-based tokenization

- A very simple way to tokenize
 - For languages that use space characters between words
 - Arabic, Cyrillic, Greek, Latin, etc., based writing systems
 - Segment off a token between instances of spaces
- Unix tools for space-based tokenization
 - The “tr” or “sed” commands.
 - Given a text file, output the word tokens and their frequencies
 - You may want to check UNIX for Poets by Ken Church (alternate version)

Simple Tokenization in UNIX

- Given a text file, output the word tokens and their frequencies

```
cat shakes.txt | tr -sc 'A-Za-z' '\n' | head
```

Change all non-alpha to newlines

```
THE  
SONNETS  
by  
William  
Shakespeare  
From  
fairest  
creatures  
We  
...
```

Simple Tokenization in UNIX

- Given a text file, output the word tokens and their frequencies

```
cat shakes.txt | tr -sc 'A-Za-z' '\n'
                | sort
                | uniq -c
```

Change all non-alpha to newlines

Sort in alphabetical order

Merge and count each type

```
1945 A
 72 AARON
 19 ABBESS
  5 ABBOT
... ..
 25 Aaron
  6 Abate
  1 Abates
  5 Abbess
  6 Abbey
  3 Abbot
... ..
```

More counting

Merging upper and lower case

```
cat shakes.txt | tr 'A-Z' 'a-z' | tr -sc 'A-Za-z' '\n' | sort | uniq -c
```

More counting

Merging upper and lower case

```
cat shakes.txt | tr 'A-Z' 'a-z' | tr -sc 'A-Za-z' '\n' | sort | uniq -c
```

Sorting the counts

```
cat shakes.txt | tr 'A-Z' 'a-z' | tr -sc 'A-Za-z' '\n' | sort | uniq -c | sort -n -r
```

```
23243 the
22225 i
18618 and
16339 to
15687 of
12780 a
12163 you
10839 my
10005 in
8954 d
```



What happened here?

Issues in Tokenization

- Can't just blindly remove punctuation:
 - m.p.h., Ph.D., AT&T, cap'n
 - prices (\$45.55)
 - dates (01/02/06)
 - URLs (<http://www.stanford.edu>)
 - hashtags ([#nlproc](#))
 - email addresses (someone@cs.colorado.edu)
- Clitic: a word that doesn't stand on its own: "are" in [we're](#), "*lhe*" in "*dar-lhe*"
- When should multiword expressions (MWE) be words? [New York](#), [rock 'n' roll](#)

Word-based tokenization in NLTK

```
import nltk
text= "That U.S.A. poster-print costs $12.40..."
tokens=nltk.word_tokenize(text)
print(tokens)
```

```
['That', 'U.S.A.', 'poster-print', 'costs', '$', '12.40', '...']
```

Word-based tokenization in NLTK

```
import nltk
text= "That U.S.A. poster-print costs $12.40..."
tokens=nltk.word_tokenize(text)
print(tokens)
```

```
['That', 'U.S.A.', 'poster-print', 'costs', '$', '12.40', '...']
```

```
tweet = "This is a coool #dummysmiley: :-) :-P <3 and some arrows < > -> <--"
tokens=nltk.word_tokenize(tweet)
print(tokens)
```

```
['This', 'is', 'a', 'coool', '#', 'dummysmiley', ':', ':', '-', ')', ':', '-P', '<', '3', 'and', 'some', 'arrows', '<', '>', '-', '>', '<', '--']
```

Word-based tokenization in NLTK

```
import nltk
text= "That U.S.A. poster-print costs $12.40..."
tokens=nltk.word_tokenize(text)
print(tokens)
```

```
['That', 'U.S.A.', 'poster-print', 'costs', '$', '12.40', '...']
```

```
tweet = "This is a coool #dummysmiley: :-) :-P <3 and some arrows < > -> <--"
tokens=nltk.word_tokenize(tweet)
print(tokens)
```

```
['This', 'is', 'a', 'coool', '#', 'dummysmiley', ':', ':', '-', ')', ':', '-P', '<', '3', 'and', 'some', 'arrows', '<', '>', '-', '>', '<', '--']
```

```
from nltk.tokenize import TweetTokenizer
mytkzr = TweetTokenizer()
tokens=mytkzr.tokenize(tweet)
print(tokens)
```

```
['This', 'is', 'a', 'coool', '#dummysmiley', ':', ':-)', ':-P', '<3', 'and', 'some', 'arrows', '<', '>', '->', '<--']
```

Word-based tokenization in NLTK

```
import nltk
text= "That U.S.A. poster-print costs $12.40..."
tokens=nltk.word_tokenize(text)
print(tokens)
```

```
['That', 'U.S.A.', 'poster-print', 'costs', '$', '12.40', '...']
```

```
tweet = "This is a coool #dummysmiley: :-) :-P <3 and some arrows < > -> <--"
tokens=nltk.word_tokenize(tweet)
print(tokens)
```

```
['This', 'is', 'a', 'coool', '#', 'dummysmiley', ':', ':', '-', ')', ':', '-P', '<', '3', 'and', 'some', 'arrows', '<', '>', '-', '>', '<', '--']
```

```
from nltk.tokenize import TweetTokenizer
mytkzr = TweetTokenizer()
tokens=mytkzr.tokenize(tweet)
print(tokens)
```

```
['This', 'is', 'a', 'coool', '#dummysmiley', ':', ':-)', ':-P', '<3', 'and', 'some', 'arrows', '<', '>', '->', '<--']
```

```
tweet = "Olá @manuel_silva, a cadeira de #text-mining rulezzzz. :)"
tokens=mytkzr.tokenize(tweet)
print(tokens)
```

```
['Olá', '@manuel_silva', ',', 'a', 'cadeira', 'de', '#text-mining', 'rulezzzz', '.', ':)']
```

Word-based tokenisation Issues

- Peter's friend → Peter 's friend
- what're, I'am, isn't → what are, I am, is not
- San Francisco → one or two tokens ?
- French
 - L'ensemble → one or two tokens ?

Tokenisation example: NLTK vs string

- O presidente da Renamo, Afonso Dhlakama, apelou hoje para que a polícia se mantenha longe das mesas de voto, durante as eleições gerais em Moçambique, e pediu aos eleitores que controlem o seu voto.

Tokenisation example: NLTK vs string

- O presidente da Renamo, Afonso Dhlakama, apelou hoje para que a polícia se mantenha longe das mesas de voto, durante as eleições gerais em Moçambique, e pediu aos eleitores que controlem o seu voto.
- | O | presidente | da | Renamo | , | Afonso | Dhlakama | , | apelou | hoje | para | que | a | polícia | se | mantenha | longe | das | mesas | de | voto | , | durante | as | eleições | gerais | em | Moçambique | , | e | pediu | aos | eleitores | que | controlem | o | seu | voto | .

Tokenisation example: NLTK vs string

- O presidente da Renamo, Afonso Dhlakama, apelou hoje para que a polícia se mantenha longe das mesas de voto, durante as eleições gerais em Moçambique, e pediu aos eleitores que controlem o seu voto.
- | O | presidente | da | Renamo | , | Afonso | Dhlakama | , | apelou | hoje | para | que | a | polícia | se | mantenha | longe | das | mesas | de | voto | , | durante | as | eleições | gerais | em | Moçambique | , | e | pediu | aos | eleitores | que | controlem | o | seu | voto | .
- | O | presidente | de | a | Renamo | , | Afonso | Dhlakama | , | apelou | hoje | para | que | a | polícia | se | mantenha | longe de | as | mesas de voto | , | durante | as | eleições gerais | em | Moçambique | , | e | pediu | a | os | eleitores | que | controlem | o | seu | voto | .

Alternate tokenisation strategies

- Character-based tokenisation
- Subword tokenization
 - tokens can be parts of words as well as whole words
 - Example: *unlikelyst* has 3 morphemes *un-*, *likely*, and *-est*
- <https://platform.openai.com/tokenizer>
- Experiment with different tokenizers here: [The Hugging Face course](#)

Subword tokenization

- Three common algorithms
 - [Byte-Pair Encoding \(BPE\)](#) (Sennrich et al., 2016)
 - [Unigram language modeling tokenization](#) (Kudo, 2018)
 - [WordPiece](#) (Schuster and Nakajima, 2012) - used in BERT, similar to BPE
- They all use the data to tell us how to tokenize, and all have 2 parts
 - A token [learner](#) that takes a raw training corpus and induces a vocabulary (a set of tokens).
 - A token [segmenter](#) that takes a raw test sentence and tokenizes it according to that vocabulary

Sentence Segmentation

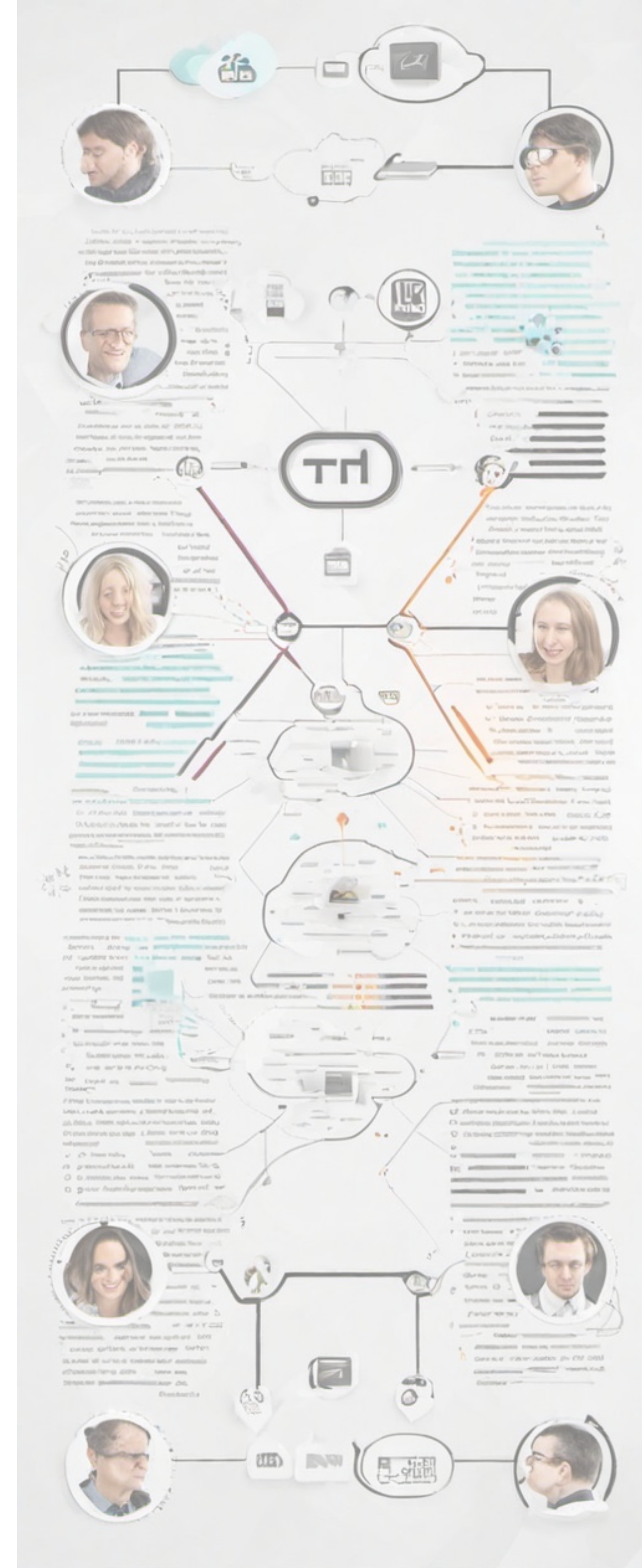
- !, ? mostly unambiguous but period “.” is very ambiguous
 - Sentence boundary
 - Abbreviations like Inc. or Dr.
 - Numbers like .02% or 4.3
- Common algorithm
 - Tokenize first: use rules or ML to classify a period as either (a) part of the word or (b) a sentence-boundary. An abbreviation dictionary can help
 - Sentence segmentation can then often be done by rules based on this tokenization

Basic text processing

Text Normalization

Case folding

Stemming



Text Normalisation

- Need to “normalise” terms (put words/tokens in a standard format)
 - `uhhuh` vs `uh-huh`; `twenty one` vs `21`; `org.` vs `organisation`
- Information Retrieval: indexed text & query terms must have same form.
 - We want to match `U.S.A.` and `USA`
- Alternative: asymmetric expansion
 - Enter: `window` Search: `window, windows`
 - Enter: `windows` Search: `Windows, windows, window`
 - Enter: `Windows` Search: `Windows`

Case folding

- Applications like IR: reduce all letters to lower case
 - Since users tend to use lower case
 - Possible exception: upper case in mid-sentence?
 - e.g., *General Motors*
 - *Fed* vs. *fed*
 - *SAIL* vs. *sail*
- For sentiment analysis, Machine Translation, Information extraction
 - Case is helpful (*US* versus *us* is important)

Stemming

- Reduce terms to stems, chopping off affixes crudely
 - language dependent
 - e.g., *automate(s)*, *automatic*, *automation* all reduced to *automat*.

Stemming

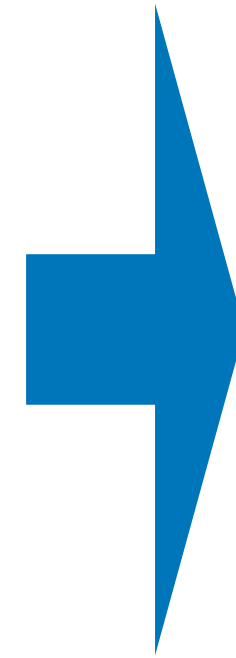
- Reduce terms to stems, chopping off affixes crudely
 - language dependent
 - e.g., *automate(s)*, *automatic*, *automation* all reduced to *automat*.

This was not the map we found in Billy Bones's chest, but an accurate copy, complete in all things -names and heights and soundings- with the single exception of the red crosses and the written notes.

Stemming

- Reduce terms to stems, chopping off affixes crudely
 - language dependent
 - e.g., *automate(s)*, *automatic*, *automation* all reduced to *automat*.

This was not the map we found in Billy Bones's chest, but an accurate copy, complete in all things -names and heights and soundings- with the single exception of the red crosses and the written notes.



Thi wa not the map we found in Billi Bone s chest but an accur copi complet in all thing name and height and sound with the singl except of the red cross and the written note .

Porter Stemmer

- Based on a series of rewrite rules run in series
 - A cascade, in which output of each pass fed to next pass
- Some sample rules:

ATIONAL	→ ATE	(e.g., relational → relate)
ING	→ ϵ	if stem contains vowel (e.g., motoring → motor)
SSES	→ SS	(e.g., grasses → grass)

- Does it work well on Portuguese?

Existing Tools

- Command line tools
- NLTK (<http://www.nltk.org>)
- TextBlob (<https://textblob.readthedocs.io>)
- Spacy (<https://spacy.io>)
- Transformers
 - Hugging face models

Command Line: why?

- It is **free**!
- It is always there for you!
- A way of working with remote computers
- Works with **large** amounts of data in an **easy** and **versatile** way
- Examples: (*how would you do those ?*)
 - get me the 1000 most common words from this 100 million words text
 - find me occurrences of a given pattern in this text
 - remove all punctuation marks from this text and convert it to lowercase
 - compare two similar lists

Text Processing

Unix Standard Tools

- Simple commands
 - cat, paste
 - wc
 - head, tail
 - cut
 - sort, uniq
 - tr
 - diff, comm
- Use regular expressions
 - grep
 - sed
 - awk
 - perl
 - ...

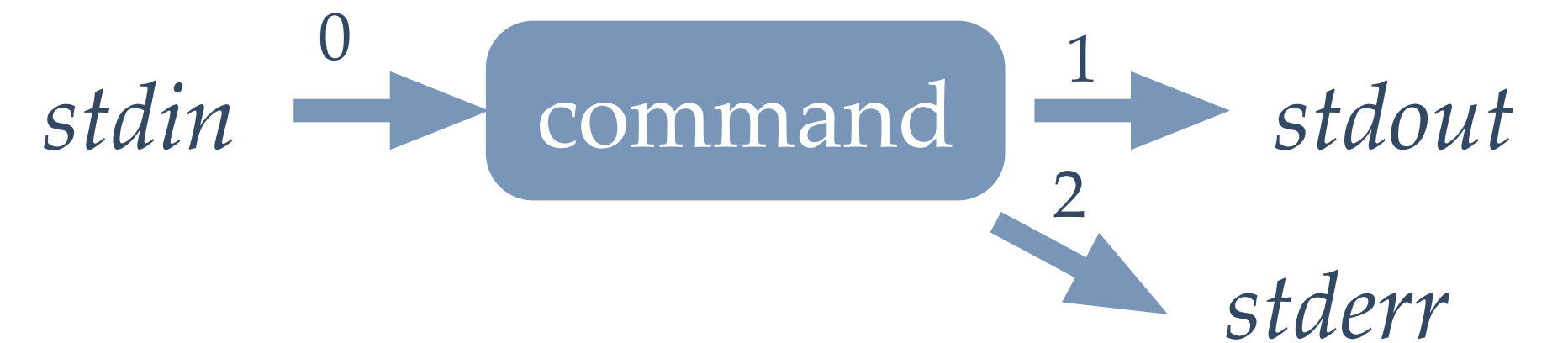
Shell Redirection: the basics

- default is *stdout* (screen)
 - `ls -l`



Shell Redirection: the basics

- default is *stdout* (screen)
 - `ls -l`
- redirecting to a file
 - `ls -l > report.txt`
 - `echo "good bye" >> report.txt`



Shell Redirection: the basics

- default is *stdout* (screen)

- ▶ `ls -l`

- ▶ redirecting to a file

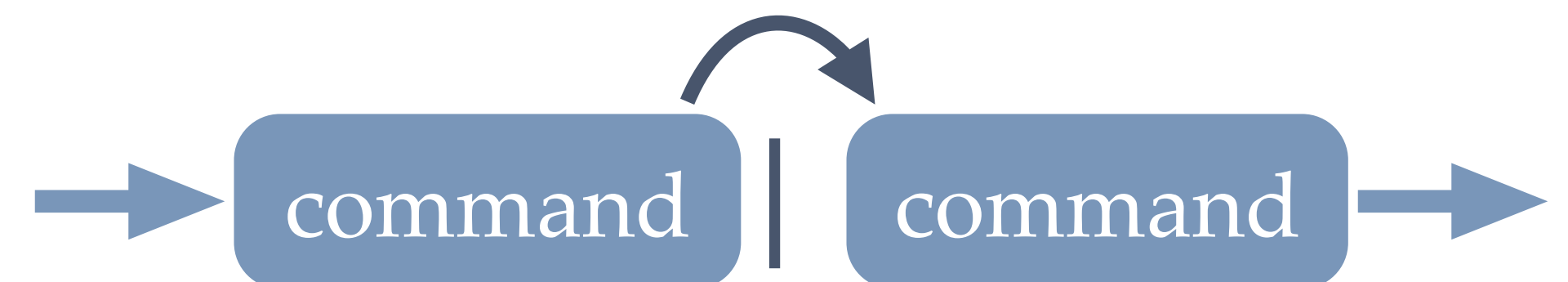
- ▶ `ls -l > report.txt`

- ▶ `echo "good bye" >> report.txt`

- ▶ redirecting to another command (pipes)

- ▶ `ls -l | wc -l`

- ▶ `echo "how many ?" | wc`



Some notes about character encodings

- It may be the case that expressions, such as

`"[a-z]", "[A-Z]", "[[:lower:]]", or "[[:upper:]]"`

don't work as expected in your operating system. If that happens, please execute one of the following commands

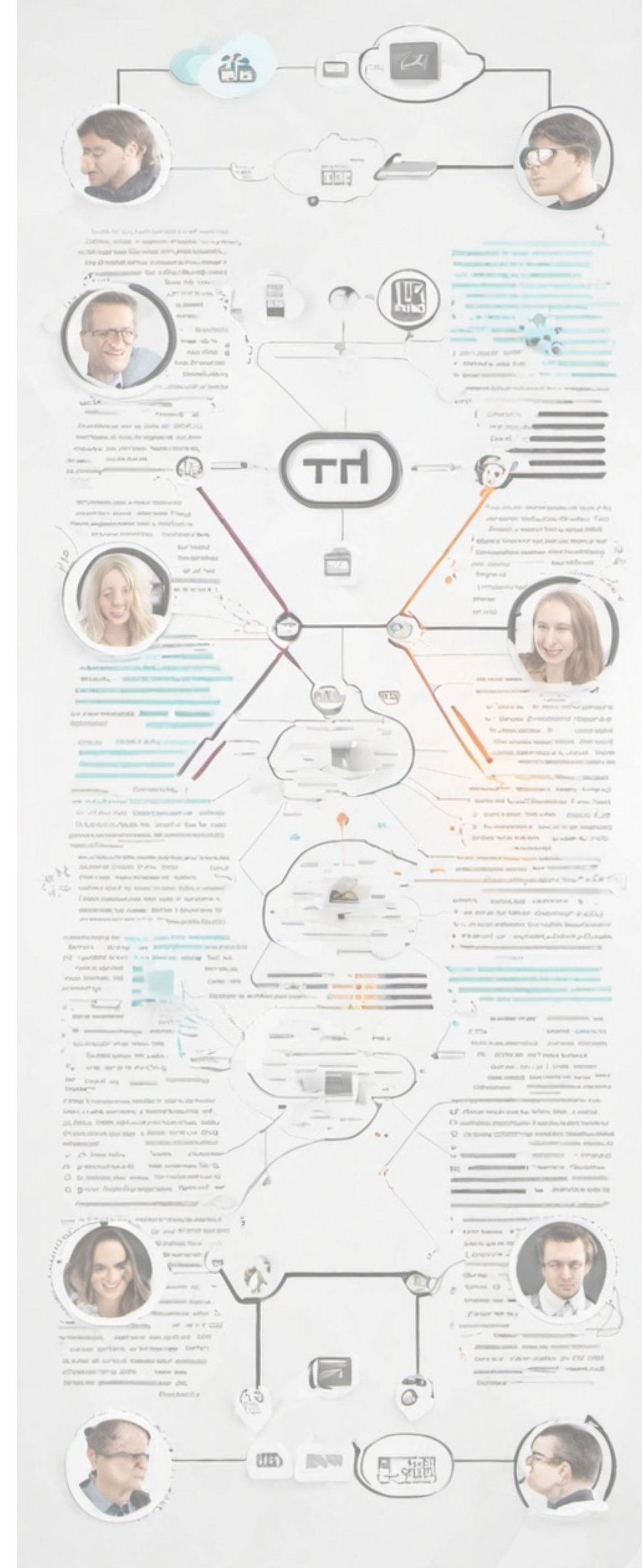
- `LANG="UTF-8"`
 - If you want to use expressions, like `"[a-z]"` and `"[A-Z]"`
- `LANG="pt_PT.UTF-8"`
 - If you want to use expressions, like `"[[:lower:]]"` and `"[[:upper:]]"`

Hand's on

- <https://github.com/HLT-ISCTE/Text-Mining>

Document Representation

Text as vectors



Document Representation and Features

- Irregular and implicitly structured representation
 - Is transformed into an explicitly structured representation
- Feature-based representation
 - Documents are represented by a set of features

Commonly Used Features

- Characters
 - Enabling to recognise e.g. morphological features.
The so called bigrams, trigrams, ... represent sequences of 2, 3, ... characters
- Words
 - Often the term word-level tokens is used instead. Tokens can be annotated (e.g. with labels representing noun, verb etc.).
 - Bag-of-words representation exploits words, but the order is ignored.
 - Word stem represents a group of related words stripped of a suffix
- Terms
 - May represent single words or multiword units, such as “White House”
- Concepts
 - For example, the concept identifier “car” can represent different words in the text, such as automobile, car, sport-car etc. Concepts are useful to represent synonyms and help to resolve polysemy

High dimensionality

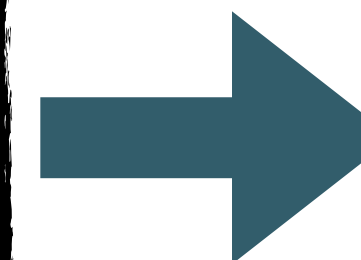
- Structured representations of natural language documents leads usually to very **large number of features**
 - For instance, one small collection of Reuters of 15,000 documents contains 25,000 non-trivial features (word stems)
- Some algorithms do not deal very well with large numbers of features and hence it is **necessary to employ feature reduction** techniques
- A related problem is **feature sparsity**
 - Each document contains only a small number of all potential features

Documents (text) as vectors

- Vector space model
 - algebraic model for representing text documents as vectors
- $|V|$ -dimensional vector space
 - V is the vocabulary
- Terms are the axes
- Documents are points in space
- Sparse vectors
 - most entries are zero

Índia volta a bater
re Em Wuhan, a cidade.
de ch O tão falado
pa de
N re ho
do es
qu m
de N
co au de
ar de
ho

Estado de
contingência: penas
mais graves para quem
não cumprir
novas **regras**.
Portugal entrou em
"situação de
contingência" devido à
Covid-19. A resolução
do **Conselho de**



	D1	D2	D3	D4
Covid-19	1		1	1
pandemia	1		2	3
regras	1			
penas	2			
empresas		2		
Portugal	1	2		
mundo			3	2
Governo	1	1		1
contingência	2			1
...				

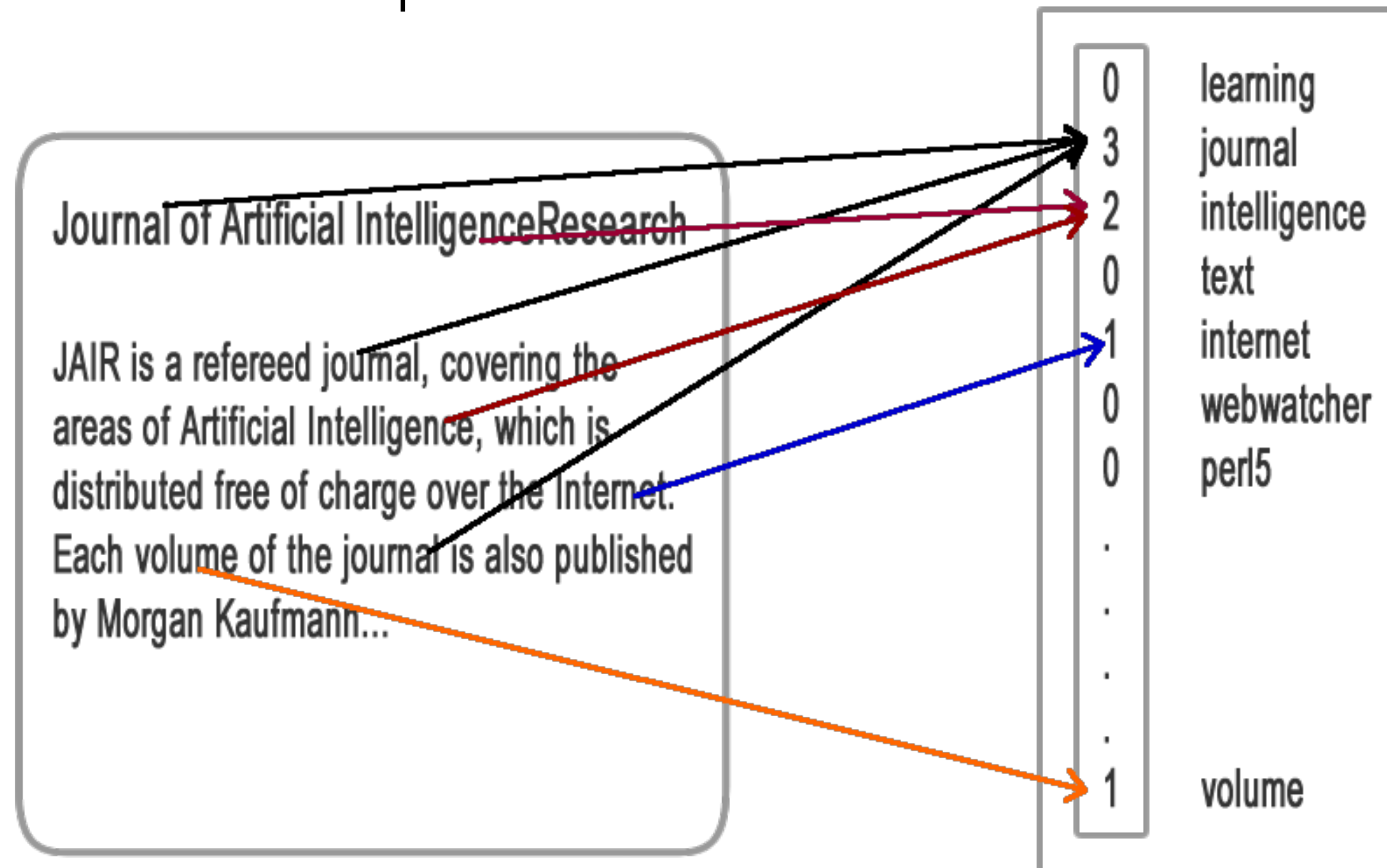
Text as vectors: Vector Space model

- Documents are treated as a “bag” of words or terms
- Any document can be represented as a vector: a list of terms and their associated weights
 - $D = \{(t_1, w_1), (t_2, w_2), \dots, (t_n, w_n)\}$
 - t_i : i -th term
 - w_i : weight for the i -th term
- Weight is a measure of the importance of terms of information content

	Document 1	Document 2	Document 3
bank	0	0	4
bass	2	4	0
commercial	0	2	2
cream	2	0	0
guitar	1	0	0
fisherman	0	3	0
money	0	1	2

Vector Space Model: bag of words

- Each document: Sparse high-dimensional vector
 - Bag-of-words document representation



Bag of Words Model

- Term frequency: $tf_{t,d}$
 - Number of times term t occurs in document d
- Order of words is not considered (bag). The following sentences have the same vector
 - The Samsung Galaxy is better than iPhone
 - The iPhone is better than Samsung Galaxy

Term weighting schemes

- Raw count ($tf_{t,d}$)
 - the elements are the raw frequency of occurrence of the word in the document
- Dampened or log-frequency
 - the elements are the logarithms of the frequency of occurrence of the word in the document
 - Dampened: $\log(tf_{t,d} + 1)$
- Log-frequency:
$$\begin{cases} 1 + \log(tf_{t,d}) & \text{if } tf_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$
- Binary
 - the elements of the vectors are either 1 or 0, where 1 indicates that the word has occurred in the document, and 0 that it has not

Weighting schemes: Inverse Document Frequency

- Considering that ...
 - Frequent terms are less informative (prepositions, articles, ...)
 - Rare terms (nouns, verbs) are more informative
- Calculate the Document Frequency (df_t)
 - Number of documents that contain the term t (df_t)
 - Inverse measure of informativeness
- and then the Inverse Document Frequency (idf_t)
 - **$\log(\text{number of documents}/df_t) \Rightarrow idf_t$**
 - There is one idf value for each term t in a collection of documents

Inverse Document Frequency

Example (taken from Jurafsky and Manning NLP course)

term	df_t	idf_t
calpurnia	1	$\log_{10}(10000000/1) = 6$
animal	100	$\dots = 4$
sunday	1 000	$\log_{10}(10000000/1000) = 3$
fly	10 000	$\dots = 2$
under	100 000	1
the	1 000 000	0

Term Frequency - Inverse Document Frequency (TF-IDF)

- General idea
 - Combine the term frequency with the inverse document frequency
 - **$TF\text{-}IDF_{t,d} = tf_{t,d} \times idf_t$**
 - Example: $TF\text{-}IDF_{t,d} = \log(1 + freq_{t,d}) \times \log(\text{total docs}/\text{docs containing } t)$
- Thus
 - Documents are represented by a real-valued vector of *tf-idf* weights, with dimensionality corresponding to the size of the vocabulary V
 - Increases with the number of occurrences of the term within a document
 - Increases with the rarity of the term in the collection

TF-IDF Example

	Document 1	Document 2	Document 3
bank	0	0	4
bass	2	4	0
commercial	0	2	2
cream	2	0	0
guitar	1	0	0
fisherman	0	3	0
money	0	1	2



	Document 1	Document 2	Document 3
bank	0	0	0,33
bass	0,08	0,12	0
commercial	0	0,08	0,08
cream	0,23	0	0
guitar	0,14	0	0
fisherman	0	0,29	0
money	0	0,05	0,08

TF-IDF Example

	Document 1	Document 2	Document 3
bank	0	0	4
bass	2	4	0
commercial	0	2	2
cream	2	0	0
guitar	1	0	0
fisherman	0	3	0
money	0	1	2

TF x IDF

$$\log(1 + \text{freq}) \times \log(N / \text{docs})$$
$$\log(1 + \mathbf{4}) \times \log(3 / \mathbf{1})$$

	Document 1	Document 2	Document 3
bank	0	0	0,33
bass	0,08	0,12	0
commercial	0	0,08	0,08
cream	0,23	0	0
guitar	0,14	0	0
fisherman	0	0,29	0
money	0	0,05	0,08

Weighting Schemes

- Local weight
 - Related to the frequency of the term in the document
- Global weight
 - Related to the number of documents that contain the term
- and, sometimes, a normalising factor

Similarity between documents

- Similarity
 - High scores for similar documents
- Another way is to calculate distance
 - Low scores for similar documents
- Generally, distances can be transformed in similarity measures

$$\text{sim}(x, y) = \frac{1}{\text{dist}(x, y)}$$

Similarity between documents

- As documents are vectors, usually, length is not important
 - Use the angle
- Small angles mean close documents
 - Maximal similarity when the angle is zero
- Cosine similarity
 - Cosine is 1 when the angle between two vectors is 0 and smaller for every other angle

$$\frac{x \cdot y}{\|x\| \|y\|}$$

Similarity between documents

Example (taken from Jurafsky and Manning NLP course)

SaS: Sense and Sensibility, PaP: Pride and Prejudice, WH: Wuthering Heights

term	log-frequency		
	SaS	PaP	WH
affection	3.06	2.76	2.30
jealous	2.00	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

term	normalized log-frequency		
	SaS	PaP	WH
affection	0.78	0.83	0.52
jealous	0.51	0.55	0.46
gossip	0.33	0	0.40
wuthering	0	0	0.58

$$\cos(\text{SaS}, \text{PaP}) \approx 0.78 \times 0.83 + 0.51 \times 0.55 + 0.33 \times 0 + 0 \times 0 \approx 0.93$$

$$\cos(\text{SaS}, \text{WH}) \approx$$

$$\cos(\text{PaP}, \text{WH}) \approx$$

Similarity between documents

Example (taken from Jurafsky and Manning NLP course)

SaS: Sense and Sensibility, PaP: Pride and Prejudice, WH: Wuthering Heights

Jane Austen

Emily Brontë

log-frequency			
term	SaS	PaP	WH
affection	3.06	2.76	2.30
jealous	2.00	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

normalized log-frequency			
term	SaS	PaP	WH
affection	0.78	0.83	0.52
jealous	0.51	0.55	0.46
gossip	0.33	0	0.40
wuthering	0	0	0.58

$\cos(\text{SaS}, \text{PaP}) \approx 0.78 \times 0.83 + 0.51 \times 0.55 + 0.33 \times 0 + 0 \times 0 \approx 0.93$

$\cos(\text{SaS}, \text{WH}) \approx$

$\cos(\text{PaP}, \text{WH}) \approx$

Some proximity measures

- **Distance measures**

- Chebyshev

$$\max_i (|x_i - y_i|)$$

- Manhattan

$$\sum_{i=1}^n |x_i - y_i|$$

- Euclidean

$$\sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- *Minkowsky*

$$\left(\sum_{i=1}^n |x_i - y_i|^N \right)^{\frac{1}{N}}$$

- **Similarity metrics**

- Dot product

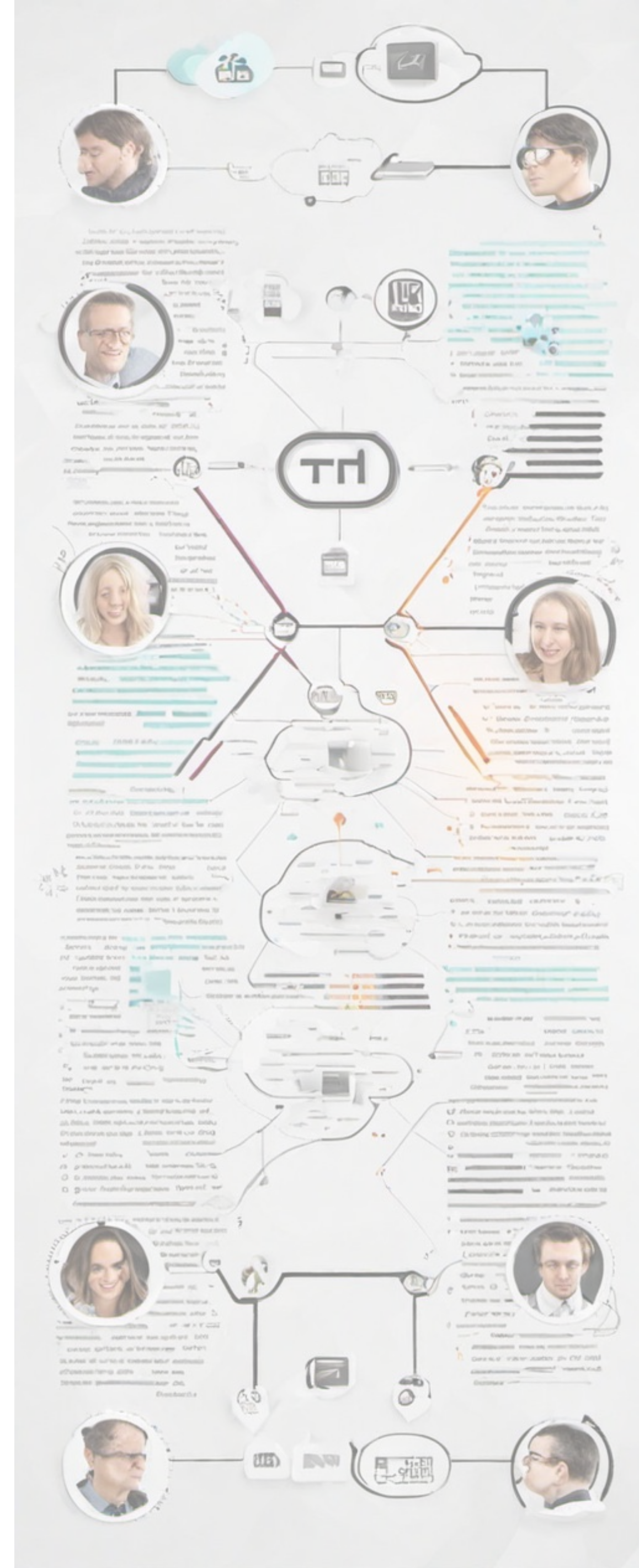
$$x \cdot y$$

- Cosine

$$\frac{x \cdot y}{\|x\| \|y\|}$$

Document Representation

Words as vectors



Words as Vectors

- A word can have a dual role in word space
 - Each word can, in principle, be a dimension word, an axis of the space (=> documents as vectors)
 - But each word can be also a vector in that space (=> words as vectors)
 - These vectors are meaningless on their own, but semantically similar words have similar vectors (and vector similarities are easy to compute, as we have seen)

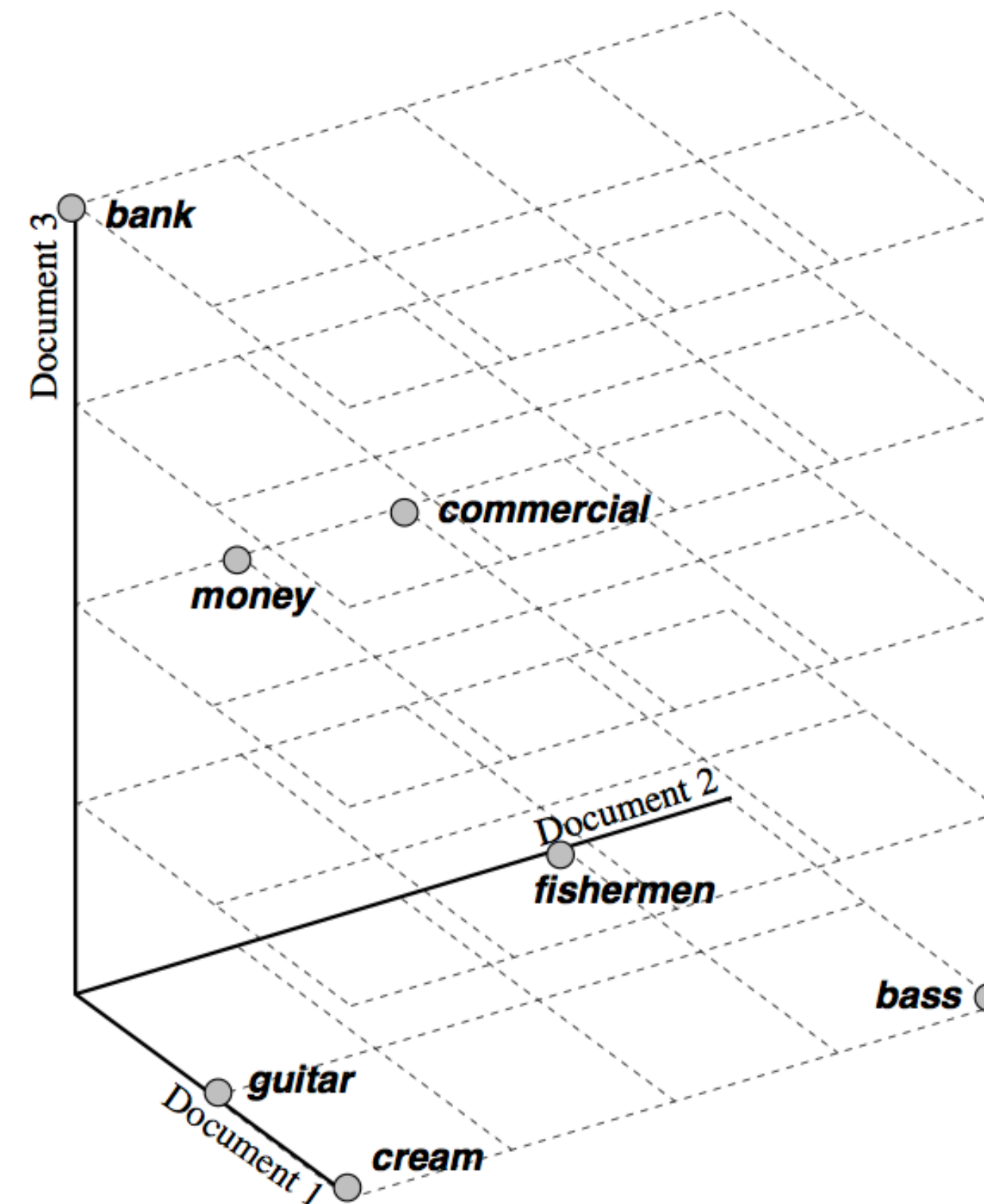
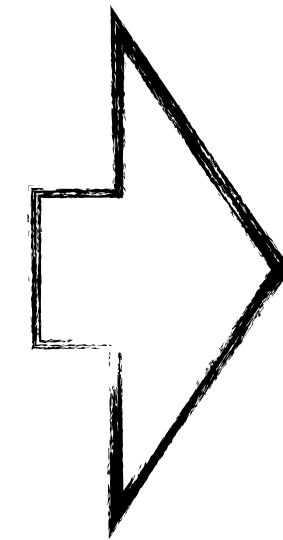
	d_1	d_2	$\dots$	d_m
w_1				
w_2				
$\dots$			m_{ij}	
w_n				

	w_1	w_2	$\dots$	w_n
w_1				
w_2				
$\dots$			f_{ij}	
w_n				

Words as Vectors

- Now documents are the axes

	Document 1	Document 2	Document 3
bank	0	0	4
bass	2	4	0
commercial	0	2	2
cream	2	0	0
guitar	1	0	0
fisherman	0	3	0
money	0	1	2



Words as Vectors

- We can take this concept a little further
 - instead of axes = documents, we consider the general concept of “context” (which can still be given by documents; but it can also be the local window around a word, or even words)

Words as Vectors

- “You shall know a word by the company it keeps” (Firth)
- Distributional Semantics
 - Take a word and its contexts. By looking at a word's context, one can infer its meaning

tasty *tnassiorc*
greasy *tnassiorc*
tnassiorc with butter
tnassiorc for breakfast

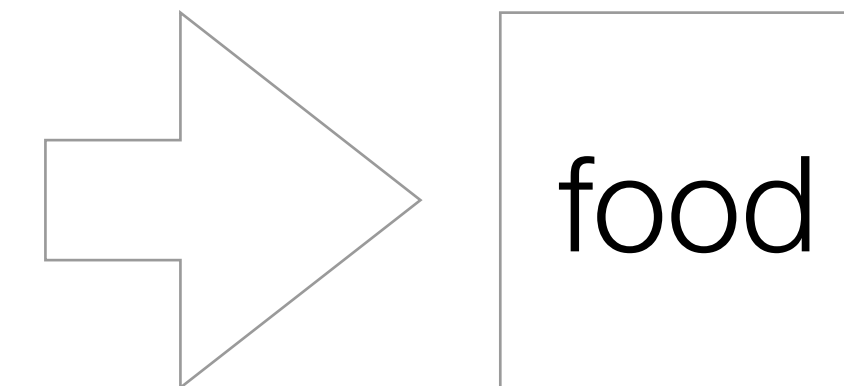
He filled the *wampimuk*,
passed it around and we all drunk some

Words as Vectors

- “You shall know a word by the company it keeps” (Firth)
- Distributional Semantics
 - Take a word and its contexts. By looking at a word's context, one can infer its meaning

tasty *tnassiorc*
greasy *tnassiorc*
tnassiorc with butter
tnassiorc for breakfast

He filled the *wampimuk*,
passed it around and we all drunk some

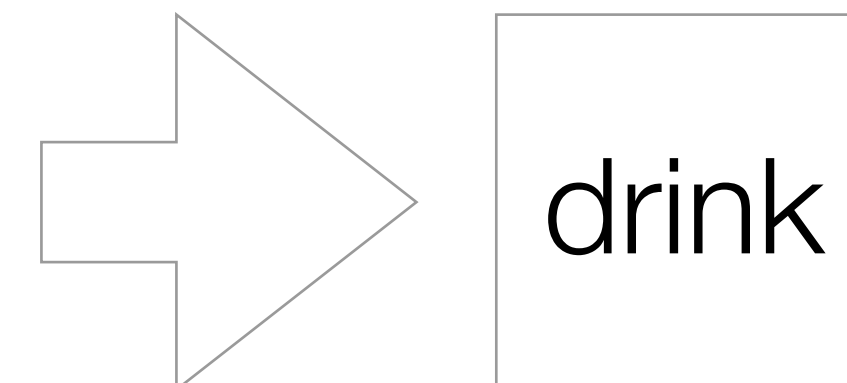
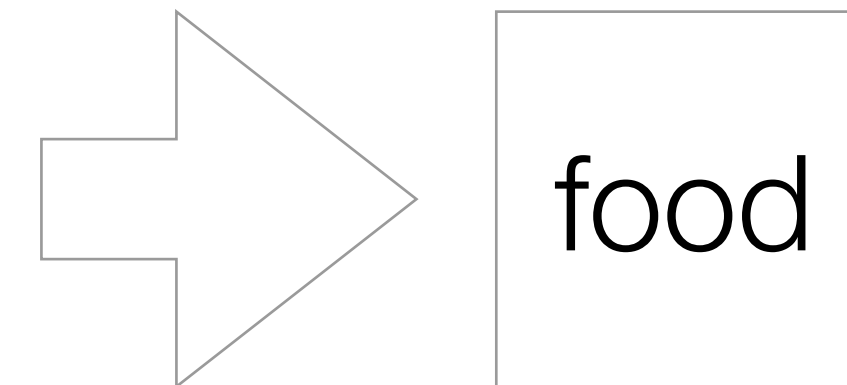


Words as Vectors

- “You shall know a word by the company it keeps” (Firth)
- Distributional Semantics
 - Take a word and its contexts. By looking at a word's context, one can infer its meaning

tasty *tnassiorc*
greasy *tnassiorc*
tnassiorc with butter
tnassiorc for breakfast

He filled the *wampimuk*,
passed it around and we all drunk some



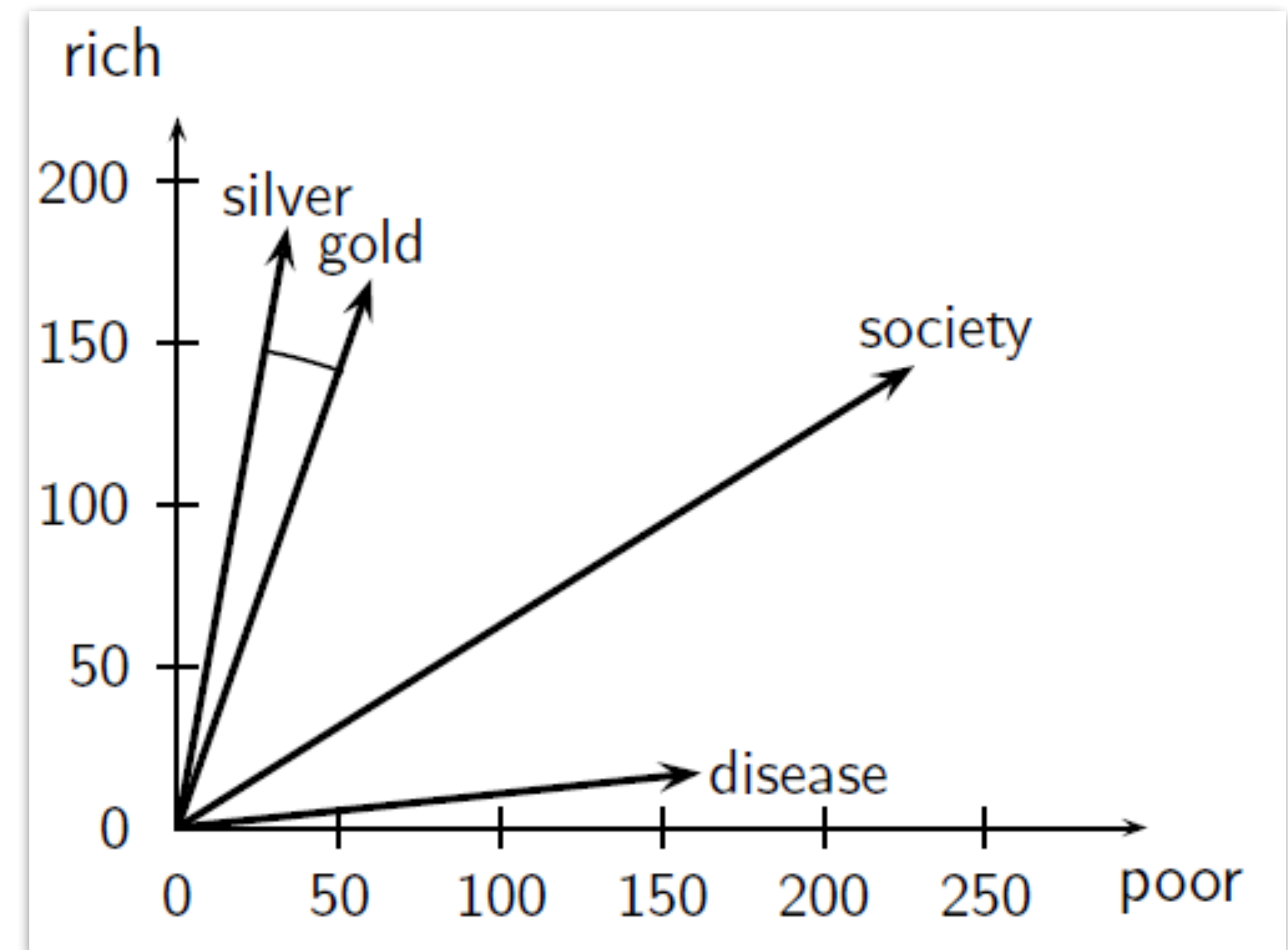
Words as Vectors

- Example:
 - Corpus = English Wikipedia
 - 2 dimensions (poor and rich)
 - Co-occurrence defined as occurrence within $k = 10$ words of each other
 - $\text{cc}(\text{rich}, \text{silver}) = 186$
 - $\text{cc}(\text{poor}, \text{silver}) = 34$
 - $\text{cc}(\text{rich}, \text{disease}) = 17$
 - $\text{cc}(\text{poor}, \text{disease}) = 162$
 - $\text{cc}(\text{rich}, \text{society}) = 143$
 - $\text{cc}(\text{poor}, \text{society}) = 228$

Words as Vectors

- Similarity between two words: usually measured with the cosine of the angle between them.
 - Small angle: silver and gold are similar.

- $\text{cc}(\text{rich}, \text{silver}) = 186$
- $\text{cc}(\text{poor}, \text{silver}) = 34$
- $\text{cc}(\text{rich}, \text{disease}) = 17$
- $\text{cc}(\text{poor}, \text{disease}) = 162$
- $\text{cc}(\text{rich}, \text{society}) = 143$
- $\text{cc}(\text{poor}, \text{society}) = 228$



Words as Vectors

- Up to now we used two dimension words: **rich** and **poor**
- Now do it for a very large number of dimension words (hundreds or thousands)
- Simple model
 - Build co-occurrence matrix F (words $\times$ words), in which f_{ij} = frequency of the co-occurrence (w_i, w_j)
- Two words are neighbours if they co-occur
 - The number of times that w_1 and w_2 occur in a corpus is their co-occurrence count
 - This count can be done considering:
 - that w_1 and w_2 occur in the same sentence or document
 - a linguistic relationship with each other (e.g., w_1 is a modifier of w_2) or
 - a distance of at most k words between w_1 e w_2 (where k is a parameter)

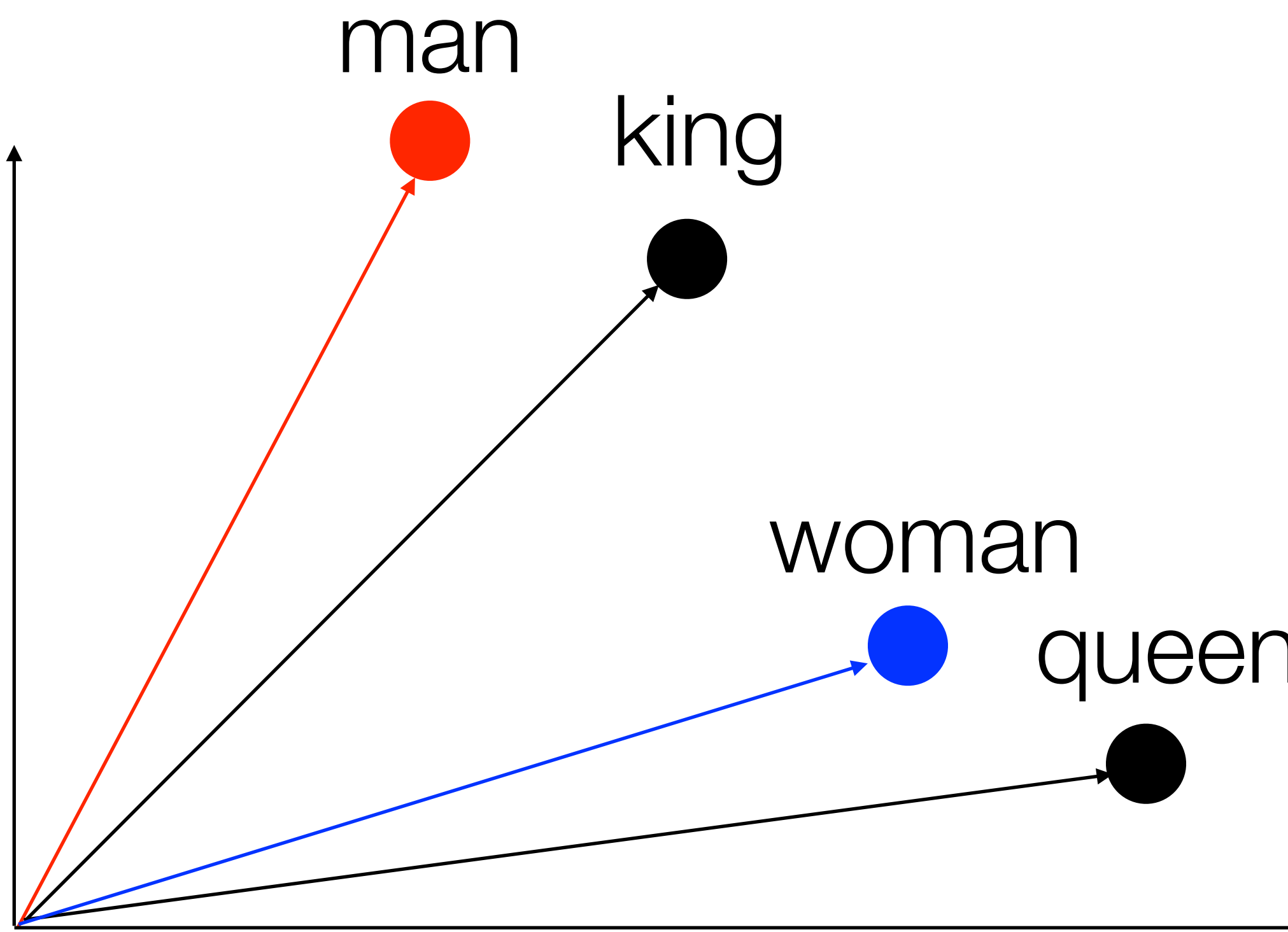
Words as Vectors

- We can compute now the nearest neighbours of any word in the word space.
- Nearest neighbors of “**silver**”: 1.000 silver / 0.865 bronze / 0.842 gold / 0.836 medal / 0.826 medals / 0.761 relay / 0.740 medalist / 0.737 coins / 0.724 freestyle / 0.720 metre / 0.716 coin / 0.714 copper / 0.712 golden / 0.706 event / 0.701 won / 0.700 foil / 0.698 Winter / 0.684 Pan / 0.680 vault / 0.675 jump
- Nearest neighbors of “**disease**”: 1.000 disease / 0.858 Alzheimer / 0.852 chronic / 0.846 infectious / 0.843 diseases / 0.823 diabetes / 0.814 cardiovascular / 0.810 infection / 0.807 symptoms / 0.805 syndrome / 0.801 kidney / 0.796 liver / 0.788 Parkinson / 0.787 disorders / 0.787 coronary / 0.779 complications / 0.778 cure / 0.778 disorder / 0.778 Crohn / 0.773 bowel

Words as Vectors

Algebraic relations

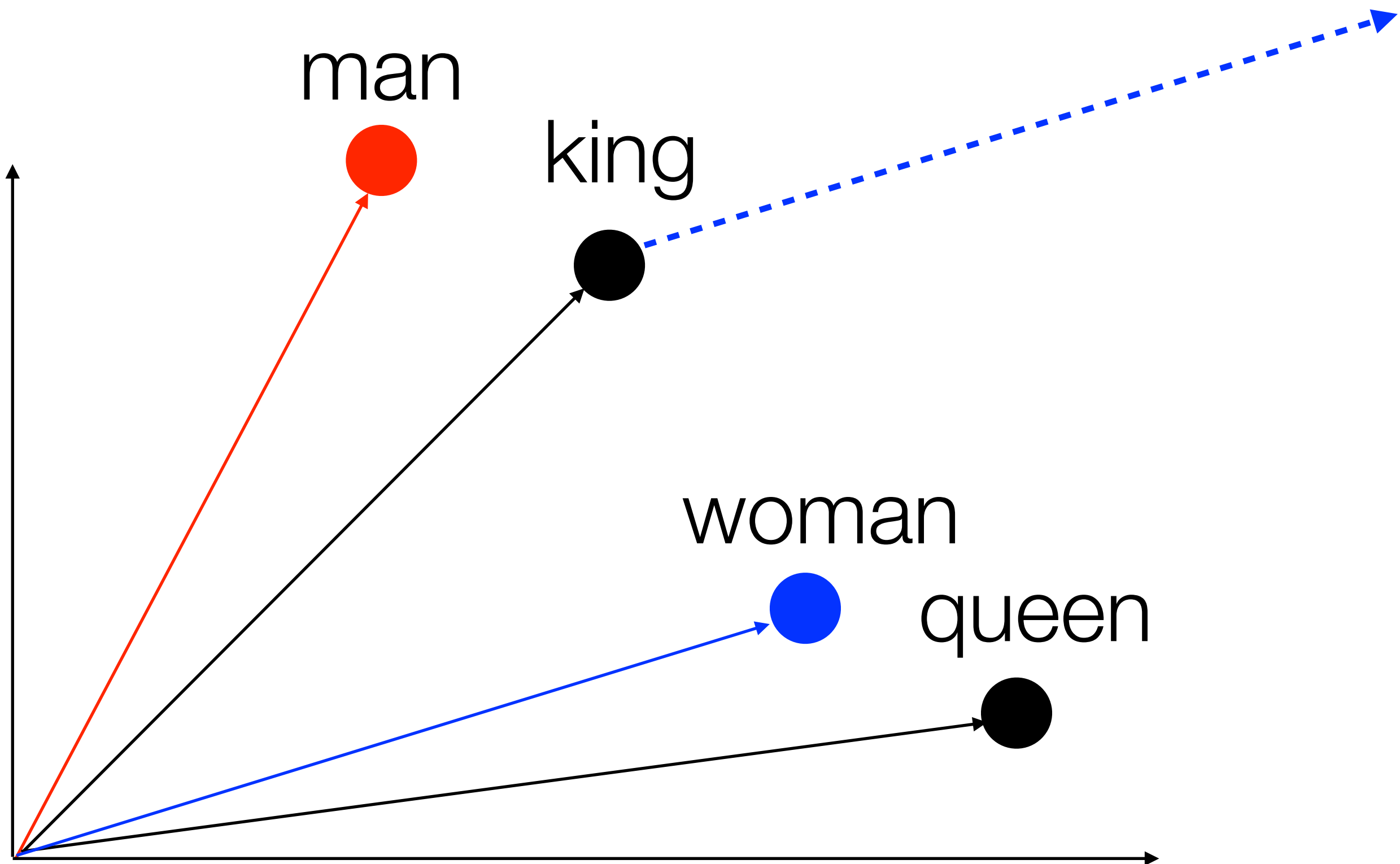
- king + woman - man ~ queen
- Lisbon - Portugal + France ~ Paris



Words as Vectors

Algebraic relations

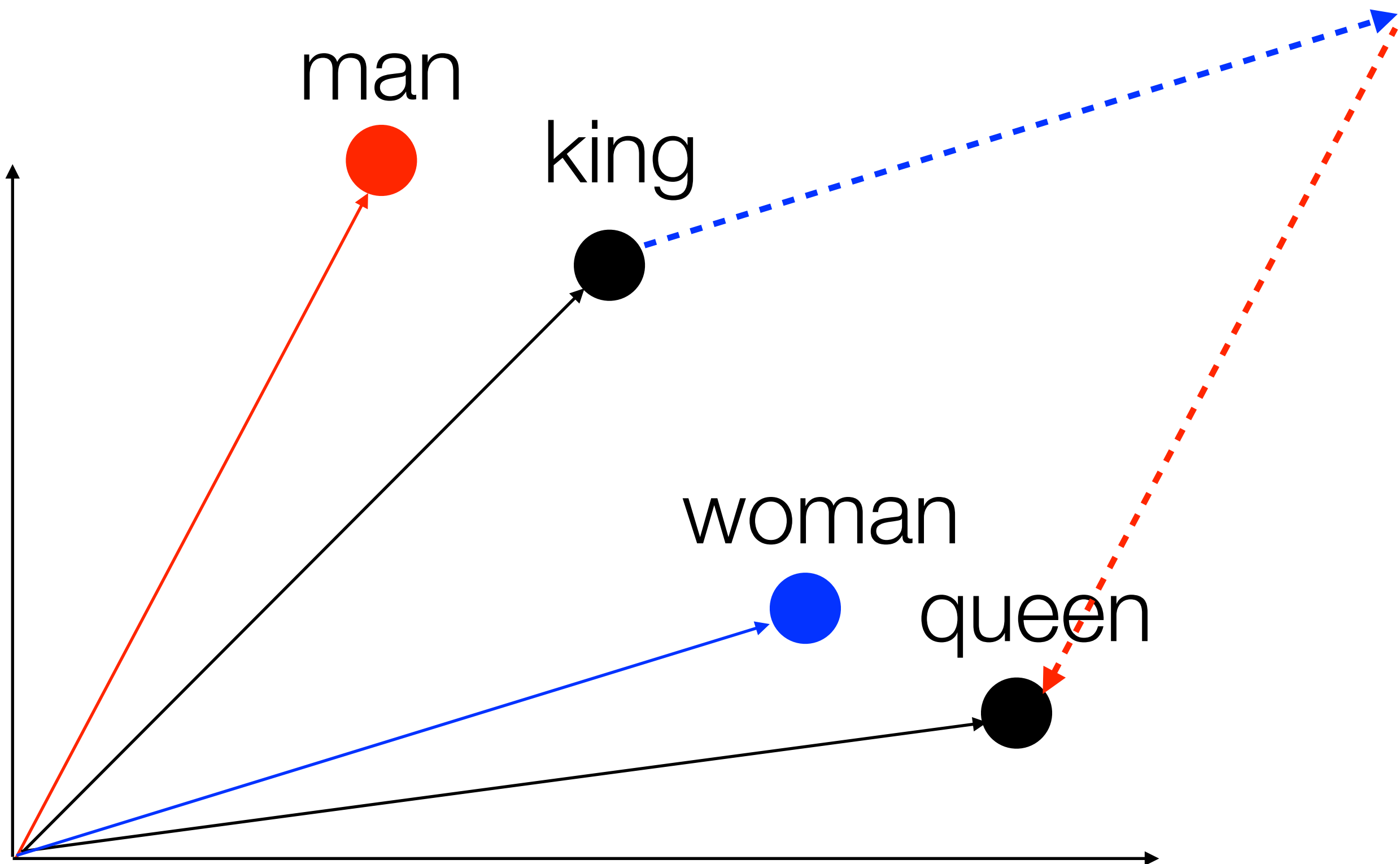
- king + woman - man ~ queen
- Lisbon - Portugal + France ~ Paris



Words as Vectors

Algebraic relations

- king + woman - man ~ queen
- Lisbon - Portugal + France ~ Paris



Words as Vectors

- Up to now
 - Very high-dimensional space with a large number of vectors represented in it
- Do
 - Dimensionality reduction

Embeddings

- “I’m afraid of bats” vs. “baseballs bats are heavy”
- Static embeddings
 - The resulting vector is the same, independently of the context of occurrence
 - Both occurrences of “bats” will have the [same vectorial representation](#)
 - Examples: word2vec, fastText, GLoVe
- Contextual embeddings
 - The resulting vector is affected by the context of occurrence
 - Each occurrence of “bats” will have a [specific vectorial representation](#)
 - Examples: ELMo, BERT

References

- [Wordspace] H. Schütze (1993), Word space, in Proceedings of the 1993 Conference on Advances in Neural Information Processing Systems, 895–902
- D. Widdows (2004), Geometry and Meaning
- M. Sahlgren (2006), The Word-Space Model, PhD Thesis
- R. Ribeiro (2011), Summarizing Spoken Documents: avoiding distracting content, PhD Thesis